

A PROJECT REPORT ON

Gesture Controlled Vehicle

**Submitted in partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology**

In

ELECTRONICS & COMMUNICATION ENGINEERING

Submitted By

<<Student_1 Anjali Shaw >>

University Roll No. 12020009002074

<<Student_2 Tuhina Roy Chaudhury >>

University Roll No. 12020009002081

<<Student_3 Shruti Das >>

University Roll No. 12020009002075

<<Student_4 Riya Kundu >>

University Roll No. 12020009002076

Under the guidance of

PROF. ARPITA DAS

Department of Electronics & Communication Engineering



UNIVERSITY OF ENGINEERING & MANAGEMENT, KOLKATA

University Area, Plot No. III – B/5, New Town, Action Area – III, Kolkata – 700160

CERTIFICATE

This is to certify that the project titled **Gesture Controlled Vehicle** submitted by Anjali Shaw [Team Leader], Tuhina Roy Chaudhury, Shruti Das, Riya Kundu, Students of UNIVERSITY OF ENGINEERING & MANAGEMENT, KOLKATA, as submitted for the partial fulfilment of requirements for the degree of Bachelor of Technology in Electronics & Communication Engineering, is a bona fide work carried out by them under the supervision and guidance of **Prof. Arpita Das** during 7th Semester of academic session of 2023. The content of this report has not been submitted to any other university or institute for the award of any other degree.

Prof. Arpita Das

Assistant Professor

Department of ECE

UEM Kolkata

Prof. Abir Chatterjee

Head of the Department

Department of ECE

UEM Kolkata

Abstract

The Gesture-Controlled Vehicle (GCV) represents a groundbreaking approach to human-vehicle interaction, leveraging advanced sensor technologies and innovative algorithms to interpret and respond to user gestures for driving and control commands. The GCV system utilizes a combination of sensors, such as infrared, depth cameras, and other gesture recognition technologies, strategically placed within the vehicle's interior. These sensors capture and interpret specific hand and body movements, translating them into actionable commands for steering, acceleration, braking, and other vehicle functions. This technology offers several advantages, including enhanced user engagement, reduced physical clutter within the vehicle, and the potential for improved safety by minimizing distractions. The fusion of gesture control with other emerging technologies sets the stage for a future where vehicles not only respond to our commands but also understand our gestures, paving the way for a more connected, user-friendly, and automated driving experience.

Keywords:

Human-vehicle interaction, Automation, Sensors.

Acknowledgements

We would like to take this opportunity to thank and acknowledge with due courtesy everyone whose cooperation and encouragement throughout the ongoing course of this project remains invaluable to us.

We are sincerely grateful to our guide Prof. Arpita Das of the Department of Electronics & Communication Engineering, UEM, Kolkata, for her wisdom, guidance and inspiration that helped us to go through with this project and take it to where it stands now.

We would also like to express our sincere gratitude to Prof. Abir Chatterjee, HOD, Electronics & Communication Engineering, UEM, Kolkata and all other departmental faculties for their ever-present assistance and encouragement.

Anjali Shaw

Tuhina Roy Chaudhury

Shruti Das

Riya Kundu

Table of Contents

Topics	Page Number
1. Introduction	7-8
2. Traditional Driving VS Automatic Driving	9-10
3.Components Required	11-12
● Components Description	13-18
• Arduino Nano	13
• NRF24LO1	13-14
• NRF Adapter	14
• MPU6050 Module	15
• L298N Motor Driver Module	15
• 7-12 DC Volt Batteries	15-16
• 4WD Car kit	16-17
• Breadboards	17
• Jumper Wires	18
4. Working Principle	19
5. Design Methodology	20-24
• Circuit Diagram	20-21
• Connections	22-23
• Circuit Pictures	24
6. Workflow	25
7. Working	26-27
8. Working code	28-34
9. Problems faced	35
10. Conclusion	36
11. Bibliography	37

LIST OF FIGURES

Sl No.	Figure No.	Topics	Page Number
1.	Figure 1	ARDUINO NANO	13
2.	Figure 2	NRF24L01	14
3.	Figure 3	NRF ADAPTER	14
4.	Figure 4	MPU6050 MODULE	15
5.	Figure 5	L298N MOTOR	15
6.	Figure 6	7-12 DC BATTERY	16
7.	Figure 7	4WD CAR KIT	17
8.	Figure 8	BREADBOARD	17
9.	Figure 9	JUMPER WIRES	18
10.	Figure 10	C.D TRANSMITTER	20
11.	Figure 11	C.D RECIEVER	21
12.	Figure 12	TRANSMITTER	24
13.	Figure 13	RECIEVER	24
14.	Figure 14	WORKFLOW	25

LIST OF TABLES

SL. NO.	TABLE NO.	TOPICS	PAGE NUMBER
1	Table 1	Hardware Requirements	11
2	Table 2	Software Requirements	12
3	Table 3	MPU6050 to Arduino	22
4	Table 4	NRF Adapter to Arduino (Transmitter)	22
5	Table 5	NRF Adapter to Arduino (Receiver)	23
6	Table 6	L298N Motor Driver to Arduino	23

INTRODUCTION

In the relentless pursuit of innovation within the automotive industry, Gesture-Controlled Vehicles (GCVs) have emerged as a cutting-edge and transformative technology, fundamentally altering the way we interact with and control vehicles. Unlike traditional automobiles, GCVs leverage advanced sensor technologies and intricate algorithms to interpret subtle hand and body movements, providing users with an intuitive and futuristic means of commanding their vehicles.

The essence of gesture control lies in its ability to replace conventional physical interfaces, such as steering wheels, pedals, and buttons, with a touchless, responsive system that captures and interprets the gestures made by the driver or passengers. This technology not only offers a glimpse into the future of driving but also aligns seamlessly with the broader trajectory towards autonomous and connected vehicles.

The integration of infrared sensors, depth cameras, and sophisticated gesture recognition algorithms allows GCVs to recognize a diverse range of gestures, translating them into specific commands that govern various aspects of the vehicle's operation. From steering and acceleration to entertainment and climate control, the possibilities for gesture-controlled interactions within a vehicle are expansive.

Gesture-controlled vehicles have the potential to revolutionize the driving experience by making it more intuitive, engaging, and accessible. As we transition toward a future dominated by autonomous driving, GCVs serve as a bridge between traditional manual control and the automated driving systems of tomorrow. This technology not only appeals to tech enthusiasts but also contributes to a broader paradigm shift in the way we perceive and interact with our vehicles.

Here's a brief introduction to the key components and concepts associated with gesture-controlled cars:

1. Gesture Recognition Technology:

- Gesture recognition involves the use of sensors and cameras to capture and interpret specific movements or gestures made by individuals.
- Advanced algorithms analyze these gestures to understand the user's intentions and translate them into corresponding commands for the car's control system.

2. Sensors and Cameras:

- Infrared sensors, depth cameras, and other sensor technologies are commonly used to detect hand and body movements.
- These sensors are strategically placed within the car's interior to provide comprehensive coverage and accurate gesture recognition.

3. Control Commands:

- Gestures can be mapped to specific commands related to car functions, such as steering, acceleration, braking, adjusting audio volume, or changing the air conditioning settings.

- For example, a simple wave of the hand might be interpreted as a command to change the radio station, while a swiping motion could control the volume.

4. Human - Machine Interaction:

- Gesture control enhances the interaction between the driver and the vehicle's onboard systems, providing a more intuitive and potentially safer way to operate certain functions.
- It can reduce the need for physical buttons and switches, simplifying the car's interior design.

5. Challenges and Considerations:

- Reliable and accurate gesture recognition is crucial for the success of gesture-controlled cars. Environmental factors such as varying lighting conditions and the presence of other objects can pose challenges.
- Safety is a primary concern, and it's essential to ensure that gesture controls do not distract the driver or compromise vehicle safety.

6. Integration with other Technologies:

- Gesture control can be integrated with other emerging technologies, such as voice recognition and artificial intelligence, to create a more seamless and responsive driving experience.

7. Future Implications:

- Gesture control is part of the broader trend towards creating more user-friendly and interconnected smart vehicles.
- As technology advances, gesture control may become a standard feature in vehicles, offering a new level of convenience and personalization for drivers and passengers alike.

This introduction sets the stage for exploring the multifaceted aspects of gesture-controlled vehicles, delving into their technological underpinnings, the impact on user experience, safety considerations, and their role in shaping the future landscape of the automotive industry. As we navigate the exciting terrain of gesture-controlled vehicles, it becomes evident that this innovative technology is more than a mere feature—it's a catalyst for redefining the very essence of driving.

TRADITIONAL VS AUTOMATIC DRIVING

Traditional driving and gesture-controlled driving represent two distinct approaches to operating a vehicle, each with its own set of characteristics, advantages, and challenges. Let's compare these two driving methods across various aspects:

1. Control Mechanism:

- **Traditional Driving:**
 - Relies on physical controls such as steering wheels, pedals, and buttons.
 - Direct and tactile feedback through these controls.
- **Gesture Controlled Driving:**
 - Utilizes hand and body movements as input commands.
 - Offers a touchless and potentially more intuitive interface.

2. Learning Curve:

- **Traditional Driving:**
 - Widely understood and taught in traditional driving education.
 - Most drivers are familiar with the standard controls.
- **Gesture Controlled Driving:**
 - Requires users to learn and adapt to specific gesture commands.
 - The learning curve may vary based on the complexity of gestures and the system's responsiveness.

3. Safety:

- **Traditional Driving:**
 - Established safety standards and practices.
 - Drivers are accustomed to the physical feedback of controls.
- **Gesture Controlled Driving:**
 - Safety concerns related to potential distractions and misinterpretation of gestures.
 - The technology must ensure that it enhances, rather than compromises, overall safety.

4. Convenience and Accessibility:

- **Traditional Driving:**
 - Familiar and widely accessible.
 - Physical controls may be more accessible to individuals with different physical abilities.
- **Gesture Controlled Driving:**
 - Offers a potentially more convenient and futuristic interface.
 - May provide an alternative for those with physical limitations.

5. User Engagement:

- **Traditional Driving:**
 - Requires focused attention on physical controls and the road.
 - Limited interaction beyond standard driving actions.
- **Gesture Controlled Driving:**

- Allows for more natural and expressive interaction with the vehicle.
- Can enhance the overall driving experience through novel interactions.

6. Technology Integration:

- **Traditional Driving:**
 - Well-established and integrated into automotive design.
 - May lack the flexibility for rapid technological updates.
- **Gesture Controlled Driving:**
 - Represents a more advanced and futuristic approach.
 - Allows for integration with other emerging technologies, such as AI and voice recognition.

7. Adaptability to Autonomous Driving:

- **Traditional Driving:**
 - Commonly associated with manual control.
 - Transition to autonomous driving may involve a significant shift in user behaviour.
- **Gesture Controlled Driving:**
 - Can potentially ease the transition to autonomous driving by offering new modes of interaction with the vehicle.

8. Environmental Considerations:

- **Traditional Driving**
 - Standard controls are less affected by environmental factors.
- **Gesture Controlled Driving:**
 - May face challenges in diverse environments, such as low light or crowded spaces.

In summary, while traditional driving has a long history of reliability and familiarity, gesture-controlled driving represents a more modern and interactive approach. The success of gesture control in driving will depend on addressing safety concerns, user acceptance, and technological advancements in gesture recognition. Both methods are likely to coexist, with the choice depending on user preferences and the specific context of use.

COMPONENTS REQUIRED

The **Hardware and Software Components** required are:

Table 1: Hardware Requirements

S. No.	COMPONENTS	USED FOR	SPECIFICATION	QUANTITY
1.	ARDUINO NANO	TRANSMITTER & RECEIVER	Operating Voltage: 5V Input Voltage: 7-12V 22 Digital I/O Pins	2
2.	NRF24LO1+ MODULE	TRANSMITTER & RECEIVER	Frequency: 2.4Ghz; Operating Voltage: 3.3 V; Max. Operating Current: 250 mA	2
3.	NRF ADAPTER	TRANSMITTER & RECEIVER	Operating Voltage: 1.9 – 3.6V; Specific Range: 100m; Input Voltage: 5V; Operating Current: 12mA	2
4.	MPU6050 MODULE	TRANSMITTER	8 Pins; 6 Degrees of Freedom	1
5.	L298N MOTOR DRIVER MODULE	RECEIVER	Operating Voltage: 5V; Operating Current: 0-36mA; Max. Power: 25W	1
6.	4WD CAR KIT	RECEIVER	a) TT GEAR MOTORS b) WHEELS c) CAR CHASIS d) SCREWS & CONNECTORS	4 (MOTORS) & 4 WHEELS
7.	7-12V DC BATTERY	TRANSMITTER (LIPO 2S BATTERY) & RECEIVER (LIPO 3S BATTERY)	Charge Current: 0.45- 0.9A; Discharge Current: 0.9-1.8A	2
8.	BREADBOARD	TRANSMITTER & RECEIVER	-	2
9.	JUMPER WIRES	TRANSMITTER & RECEIVER	-	M-F(15) , M-M(10), F-F(5)

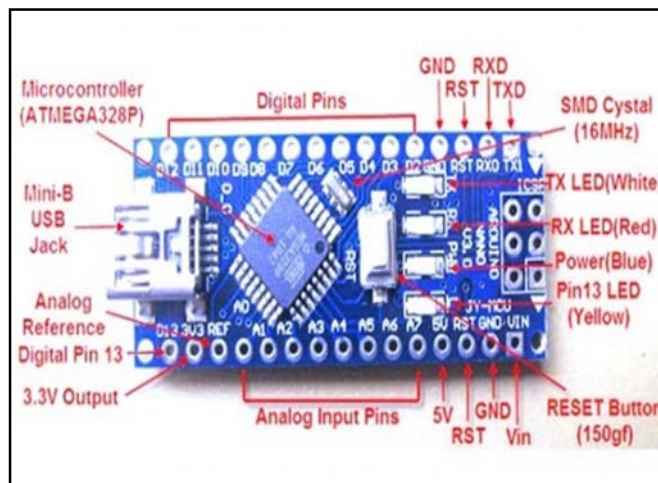
Table 2: Software Requirements

Sl.No.	COMPONENTS	DESCRIPTION
1.	Arduino IDE	Arduino provides a simple and easy-to-understand coding platform through the Arduino IDE. It is friendly to students and to those who are still new in electronics projects, simplifying coding and compiling and uploading codes to the board, eliminating the need for an external programmer or burner.

COMPONENTS DESCRIPTION

1. **Arduino Nano:** The Arduino Nano is an open-source breadboard-friendly microcontroller board based on the Microchip ATmega328P microcontroller (MCU) and developed by Arduino.cc . It consists of (PQFP) Plastic Quad Flat Pack with 32 pins, the two extra pins in Arduino Nano are analog, leaving Arduino Nano with 8 analog pins. With an Arduino Nano, you can build your own IoT-based weather data logger using the Blues Wireless Notecard and Notecarrier.

Figure 1: ARDUINO NANO



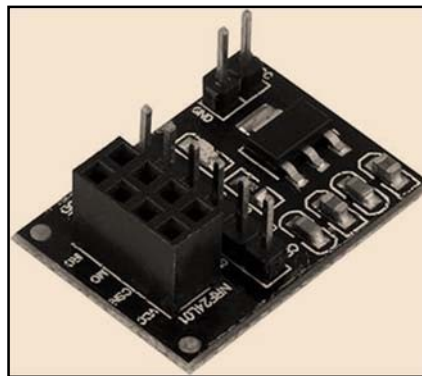
2. **NRF24LO1+ MODULE:** The NRF24L01 is a wireless transceiver module that operates in the worldwide ISM frequency band commonly used for wireless communication. The module can send and receive data using an operating radio frequency. It connects to the Arduino hardware to upload programs and communicate with them. The module can use 125 different channels which gives a possibility to have a network of 125 independently working modems in one place. Each channel can have up to 6 addresses, or each unit can communicate with up to 6 other units at the same time.

Figure 2: NRF24LO1



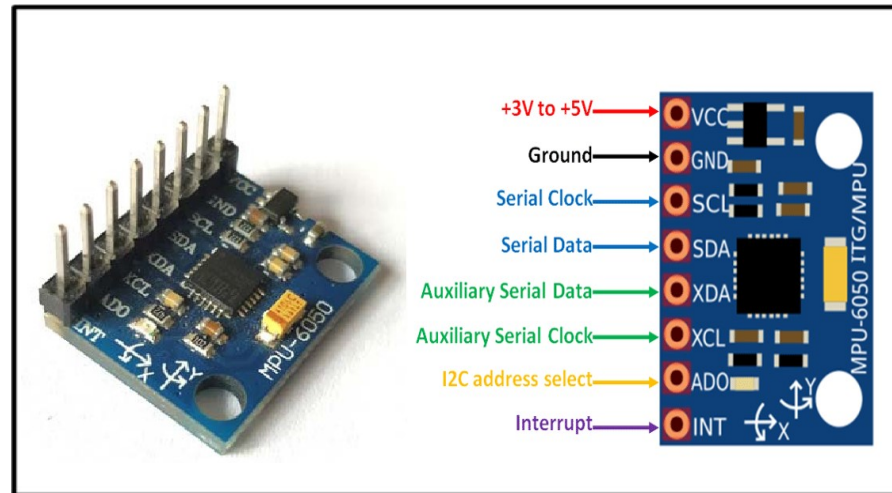
3. **NRF Adapter:** NRF Serial Adapter board allows you to plugin your nRF modules and communicate through Serial communication. They operate at a voltage range of 1.9 – 3.6V for power.

Figure 3: NRF ADAPTER



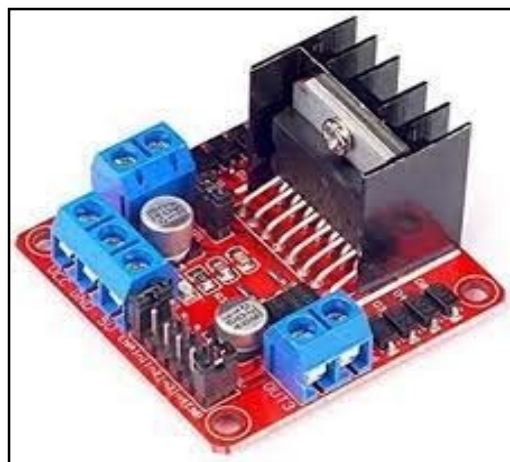
4. **MPU6050 Module:** The MPU-6050 Triple-Axis Accelerometer & Gyroscope Module is a 6-axis motion tracking device that consists of a 3-axis accelerometer and a 3-axis gyroscope.

Figure 4: MPU6050



5. **L298N MOTOR DRIVER MODULE:** The L298N is a dual H-Bridge motor driver which allows speed and direction control of two DC motors at the same time. If the microprocessor transmits a HIGH input to the motor driver, The driver will rotate the motor in one direction keeping the one pin as HIGH and one pin as LOW.

Figure 5: L298N DRIVER MOTOR

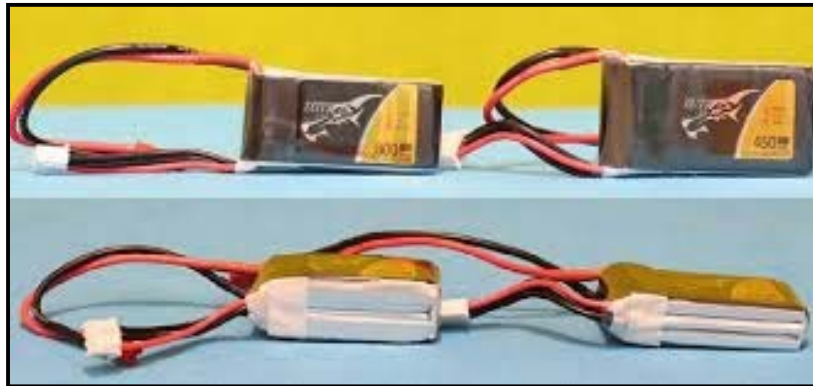


6. **7-12V DC BATTERY:**

- **LIPO 2S BATTERY:** A lithium-ion polymer battery in which a 2S LiPo means that there are 2 cells in series within the LiPo battery pack.
- **LIPO 3S BATTERY:** A 3S LiPo means that there are 3 cells in series

within the LiPo battery pack.

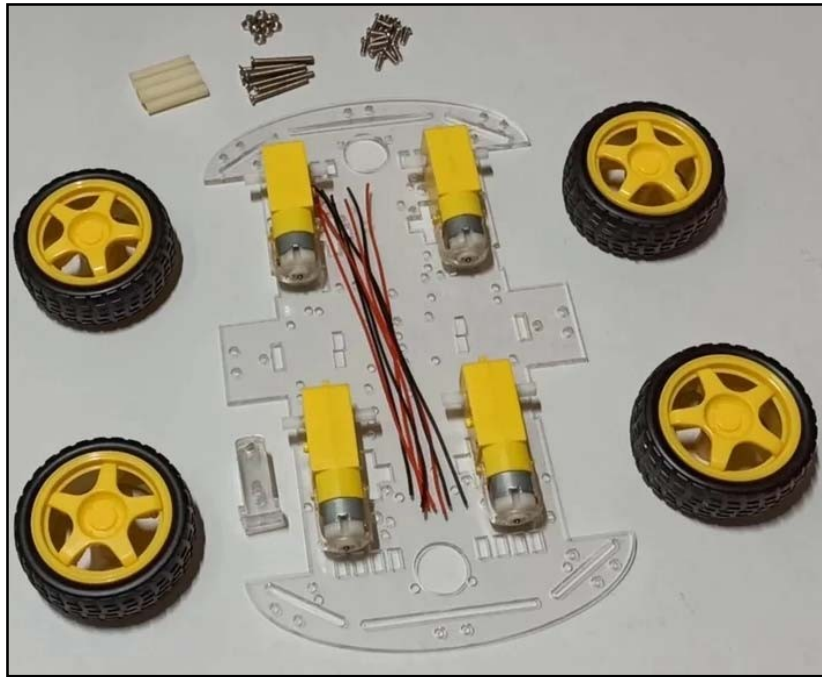
Figure 6: LIPO BATTERIES



7. 4WD CAR KIT:

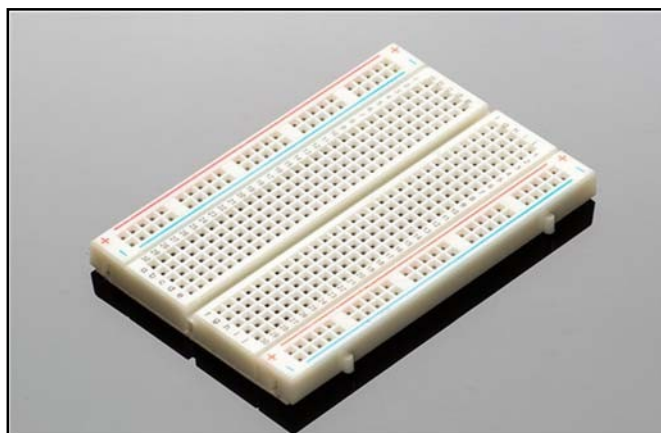
- **TT GEAR MOTORS:** The TT motor series the most popular DC motors for DIY enthusiasts. It is designed to easily incorporate the DFRobot Encoder.
- **WHEELS:** Used for the car to move.
- **CAR CHASIS:** For the architechture of the whole robotic Model.
- **SCREWS & CONNECTORS:** To connect all components and make the whole connections at place and ready to work upon.

Figure 7: 4WD CAR KIT COMPONENTS



8. **Breadboards**: A breadboard is a rectangular plastic board with a bunch of tiny holes in it. These holes let you easily insert electronic components to prototype an electronic circuit, with a battery, switch, resistor, and LED.

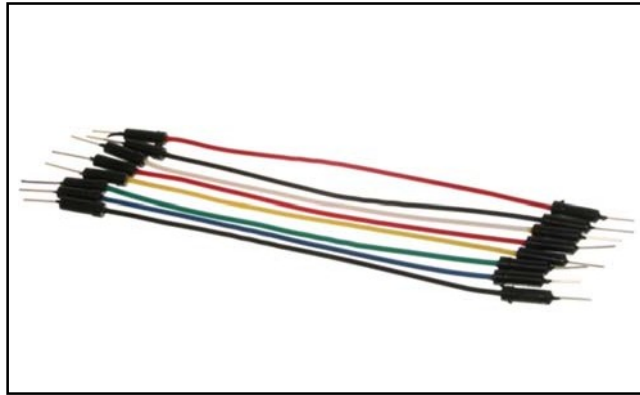
Figure 8: BREADBOARD



9. **Jumper Wires**: A jump wire is an electrical wire, or group of them in a cable, with a connector or pin at each end which is normally used to interconnect the components of a breadboard or other prototype or test circuit, internally or with

other equipment or components, without soldering.

Figure 9: JUMPER WIRES



WORKING PRINCIPLE

The transmitter section has an Arduino NANO, MPU6050 and NRF24L01.

The receiver section consists of an Arduino NANO, NRF24L01, four DC motors, and an L298N motor driver.

The main working principle is based on the sensing the wrist orientation and wireless data transmission between the NRF24 modules and running the motor using the motor driver.

The orientation of the palm controls the motion of the robot car. MPU6050 to sense the orientation of the wrist and transmits it to the Arduino in digital value. Arduino NANO continuously gets data from MPU6050. NRF24 module connected to NANO transmits this data to NRF24 module on the receiver. The circuit board of NRF24 module has a built-in antenna. The module communicates with microcontrollers.

The data is received by Arduino NANO on the receiver module which will eventually moves the DC motors based on the orientation.

DESIGN METHODOLOGY

CIRCUIT DIAGRAM:

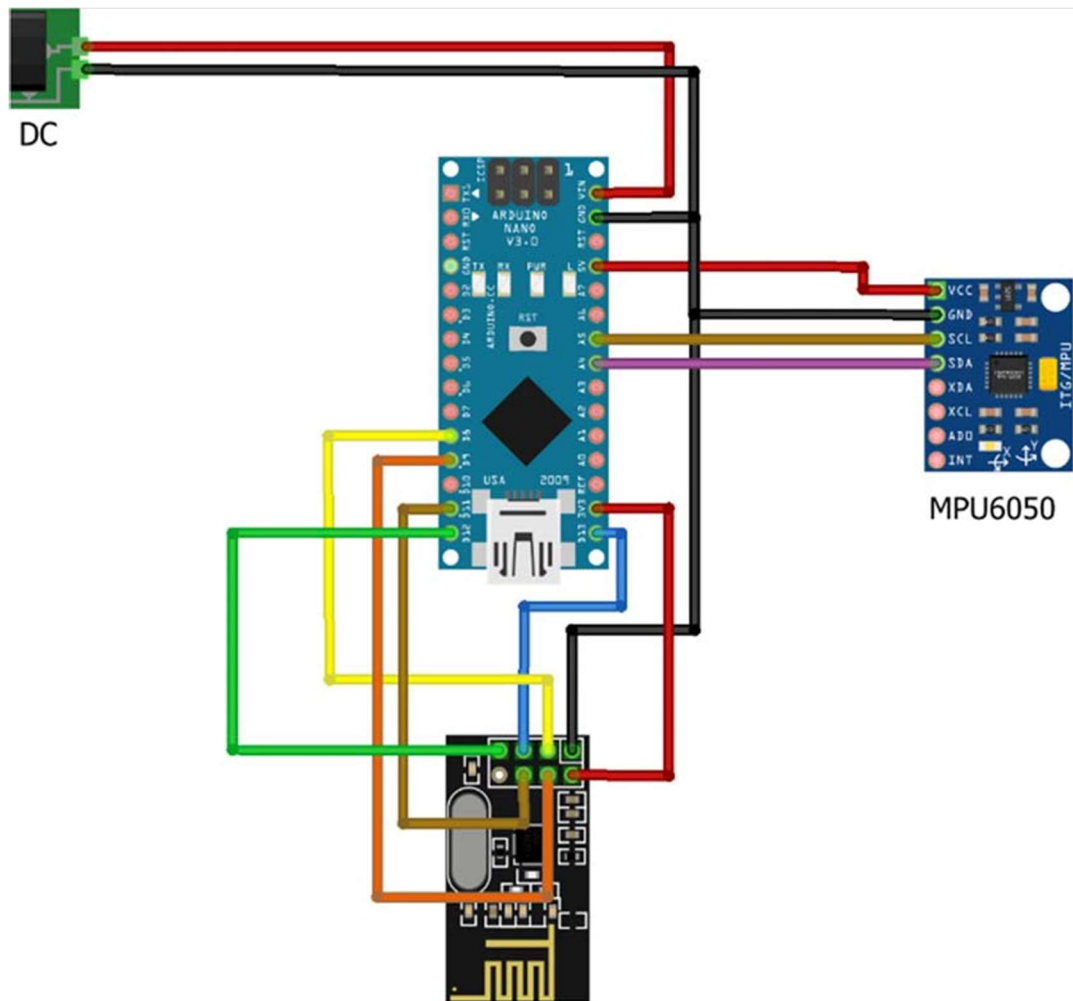


Figure 10: Circuit Diagram for Transmitter

CONNECTIONS:

A. TRANSMITTER CONNECTION

I. MPU6050 to Arduino

MPU6050	Arduino
VCC	+5
GND	GND
SDA	A4
SCL	A5

Table-3

II. NRF Adapter to Arduino

NRF Adapter	Arduino
VCC	+5
GND	GND
CE	D8
CSN	D9
SCK	D13
MO	D11
MI	D12

Table-4

B. RECEIVER CONNECTION

I. NRF Adapter to Arduino

NRF Adapter	Arduino
VCC	+5
GND	GND
CE	D8
CSN	D9
SCK	D13
MO	D11
MI	D12

Table-5

II. L298N Motor Driver to Arduino

L298N Motor Driver	Arduino
ENA	D5
IN1	D2
IN2	D3
IN3	D4
IN4	D7
ENB	D6

Table-6

CIRCUIT PICTURES:

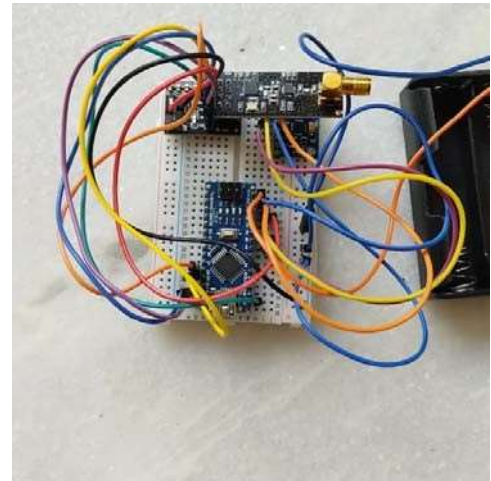
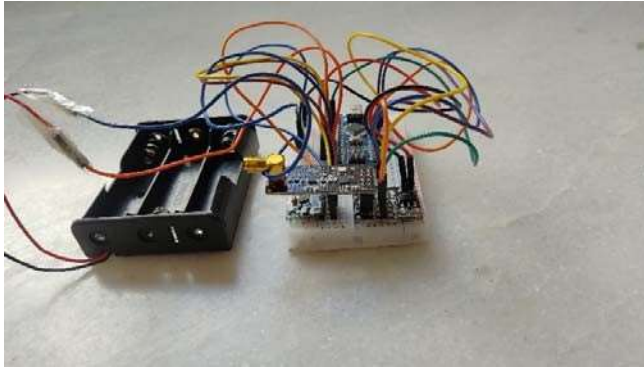


Figure 12: Transmitter

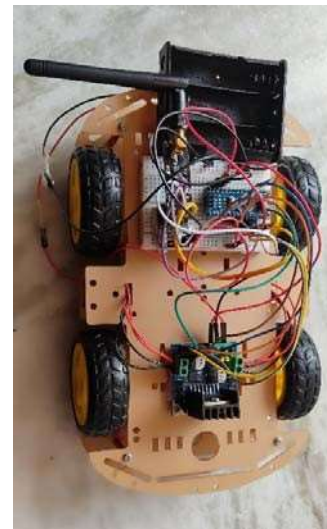
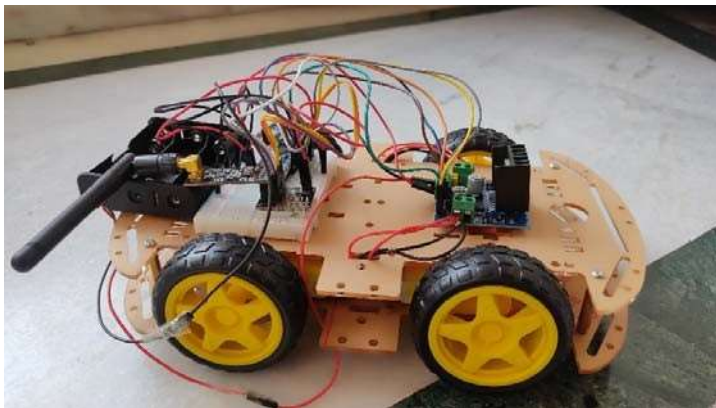


Figure 13: Receiver

WORKFLOW

MPU6050 to sense the orientation of the wrist

Transmits

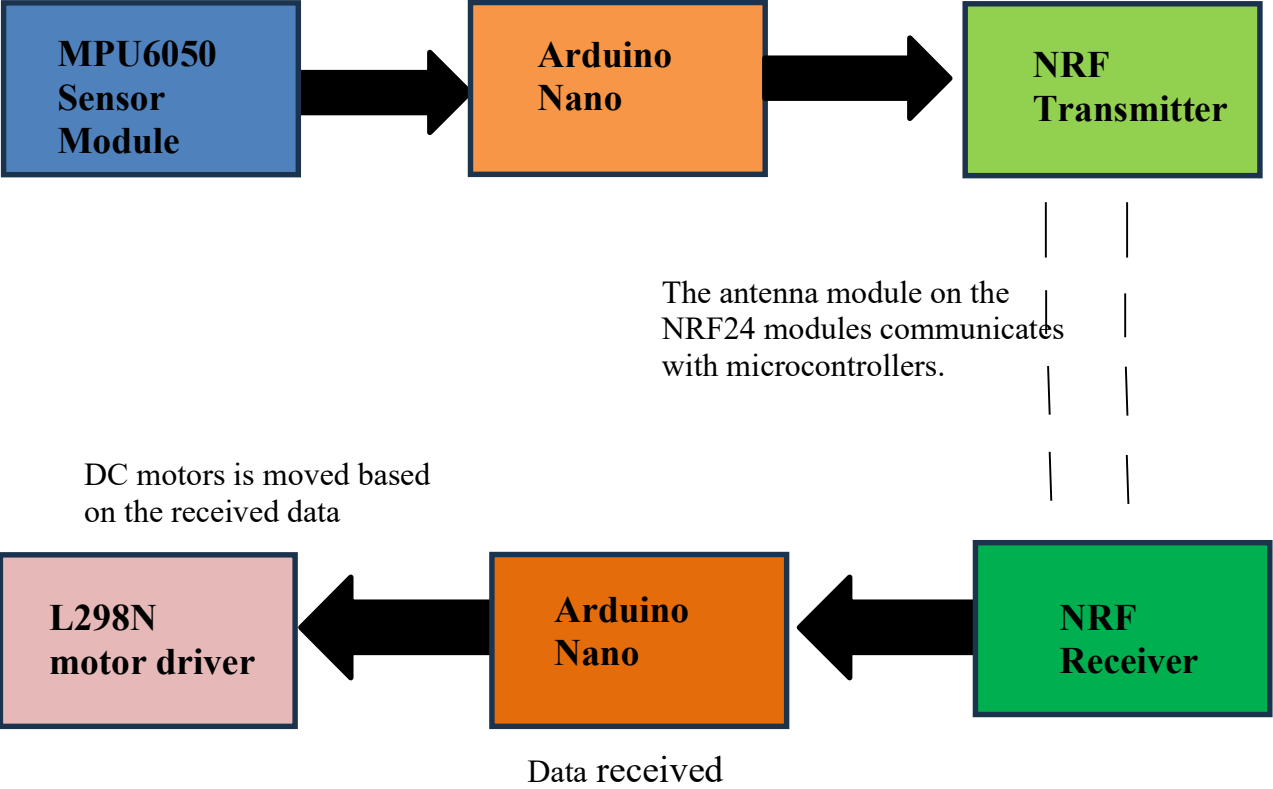


Figure 14: Workflow of the model

WORKING

MPU6050: This is a sensor module that combines an accelerometer and a gyroscope. It detects the car's orientation and movement based on hand gestures.

Arduino Nano: Serves as the microcontroller board, processing the data received from the MPU6050 and controlling the car's motors.

NRF Transceiver: Facilitates wireless communication between the controller (hand gestures) and the car.

Gesture Detection with MPU6050:

1. The MPU6050 constantly measures the car's orientation and acceleration.
2. It collects data on pitch, roll, and yaw (tilt, rotation, and direction).
3. Hand gestures create changes in these parameters, which are detected by the MPU6050.

Data Processing with Arduino Nano:

1. The MPU6050 sends the collected data to the Arduino Nano.
2. The Arduino Nano interprets these signals to recognize specific hand gestures based on predefined thresholds or patterns.
3. It processes the received data and translates it into control commands for the car's movement (forward, backward, left, right, stop).

Wireless Transmission using NRF Transceiver:

1. Once the Arduino Nano processes the gestures, it sends corresponding signals wirelessly.
2. The NRF transceiver on the car receives these signals and forwards them to the car's control system.

Car Control and Movement:

1. The car's control system interprets the received signals from the NRF transceiver.
2. It adjusts the motors and wheels according to the commands received from the hand gestures via Arduino Nano.

3. Based on the recognized gesture, the car moves in the desired direction or performs the intended action.

Implementation Details:

1. The Arduino Nano program involves libraries to read data from the MPU6050 sensor, process gestures, and control the motors.
2. The NRF transceiver modules on both the car and the hand gesture controller need compatible communication protocols for seamless data transmission.
3. Predefined gesture patterns or thresholds are set in the Arduino code to recognize specific gestures accurately.

This project amalgamates sensor data, microcontroller processing, wireless communication, and motor control to create a hand gesture-controlled car. The intricacy lies in accurately interpreting gestures and seamlessly translating them into vehicle movements.

WORKING CODE

TRANSMITTER CODE:-

```
//These are needed for MPU
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"

//These are need for RF handlling
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define PRINT_DEBUG //Uncomment this line if you want to print the MPU6050 initialization
information on serial monitor

// Arduino Wire library is required if I2Cdev I2CDEV_ARDUINO_WIRE implementation
// is used in I2Cdev.h
#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
    #include "Wire.h"
#endif

// MPU control/status vars
MPU6050 mpu;
bool dmpReady = false; // set true if DMP init was successful
uint8_t devStatus; // return status after each device operation (0 = success, != 0 = error)
uint16_t packetSize; // expected DMP packet size (default is 42 bytes)
uint8_t fifoBuffer[64]; // FIFO storage buffer
Quaternion q; // [w, x, y, z] quaternion container
VectorFloat gravity; // [x, y, z] gravity vector
float ypr[3]; // [yaw, pitch, roll] yaw/pitch/roll container and gravity vector

// RF control
const uint64_t pipeOut = 0xF9E8F0F0E1LL; //IMPORTANT: The same as in the receiver
0xF9E8F0F0E1LL
RF24 radio(8, 9); // select CE,CSN pin

struct PacketData
{
    byte xAxisValue;
    byte yAxisValue;
} data;
```

```

void setupRadioTransmitter()
{
  radio.begin();
  radio.setDataRate(RF24_250KBPS);
  radio.openWritingPipe(pipeOut);
  radio.stopListening(); //start the radio communication for Transmitter

  data.xAxisValue = 127; // Center
  data.yAxisValue = 127; // Center
}

```

```

void setupMPU()
{
  // join I2C bus (I2Cdev library doesn't do this automatically)
  #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
    Wire.begin();
    Wire.setClock(400000); // 400kHz I2C clock. Comment this line if having compilation
difficulties
  #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
    Fastwire::setup(400, true);
  #endif

  #ifndef PRINT_DEBUG
    // initialize serial communication
    Serial.begin(115200);
    while (!Serial); // wait for Leonardo enumeration, others continue immediately
    // initialize device
    Serial.println(F("Initializing I2C devices..."));
  #endif

  mpu.initialize();

  #ifndef PRINT_DEBUG
    // verify connection
    Serial.println(F("Testing device connections..."));
    Serial.println(mpu.testConnection() ? F("MPU6050 connection successful") : F("MPU6050
connection failed"));
    // wait for ready
    Serial.println(F("\nSend any character to begin DMP programming and demo: "));
    while (Serial.available() && Serial.read()); // empty buffer
    while (!Serial.available()); // wait for data
    while (Serial.available() && Serial.read()); // empty buffer again
    // load and configure the DMP
    Serial.println(F("Initializing DMP..."));

```

```

#endif

devStatus = mpu.dmpInitialize();

// make sure it worked (returns 0 if so)
if (devStatus == 0)
{
    // Calibration Time: generate offsets and calibrate our MPU6050
    mpu.CalibrateAccel(6);
    mpu.CalibrateGyro(6);

    #ifdef PRINT_DEBUG
        mpu.PrintActiveOffsets();
        // turn on the DMP, now that it's ready
        Serial.println(F("Enabling DMP..."));
    #endif
    mpu.setDMPEnabled(true);

    // set our DMP Ready flag so the main loop() function knows it's okay to use it
    #ifdef PRINT_DEBUG
        Serial.println(F("DMP ready! Waiting for first interrupt..."));
    #endif
    dmpReady = true;

    // get expected DMP packet size for later comparison
    packetSize = mpu.dmpGetFIFOPacketSize();
}
else
{
    // ERROR!
    // 1 = initial memory load failed
    // 2 = DMP configuration updates failed
    // (if it's going to break, usually the code will be 1)
    #ifdef PRINT_DEBUG
        Serial.print(F("DMP Initialization failed (code "));
        Serial.print(devStatus);
        Serial.println(F(")"));
    #endif
}
}

void setup()
{
    //This is to set up radio transmitter for rf
    setupRadioTransmitter();
}

```

```

//This is to set up MPU6050 sensor
setupMPU();
}

void loop()
{
  // if programming failed, don't try to do anything
  if (!dmpReady) return;
  // read a packet from FIFO. Get the Latest packet
  if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer))
  {
    // display Euler angles in degrees
    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

    int xAxisValue = constrain(ypr[2] * 180/M_PI, -90, 90);
    int yAxisValue = constrain(ypr[1] * 180/M_PI, -90, 90);
    data.xAxisValue = map(xAxisValue, -90, 90, 0, 254);
    data.yAxisValue = map(yAxisValue, -90, 90, 254, 0);

    radio.write(&data, sizeof(PacketData));

    #ifdef PRINT_DEBUG
    Serial.println(xAxisValue);
    Serial.println(yAxisValue);
    #endif
  }
}

```

RECEIVER CODE:-

```

#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>

#define PRINT_DEBUG //Uncomment this line if you want to print the values on serial
monitor

#define SIGNAL_TIMEOUT 500 // This is signal timeout in milli seconds.

const uint64_t pipeIn = 0xF9E8F0F0E1LL;
RF24 radio(8, 9);
unsigned long lastRecvTime = 0;

struct PacketData
{

```

```
    byte xAxisValue;  
    byte yAxisValue;  
} receiverData;
```

```
//Right motor  
int enableRightMotor=5;  
int rightMotorPin1=2;  
int rightMotorPin2=3;
```

```
//Left motor  
int enableLeftMotor=6;  
int leftMotorPin1=4;  
int leftMotorPin2=7;
```

```
void setup()  
{  
    pinMode(enableRightMotor,OUTPUT);  
    pinMode(rightMotorPin1,OUTPUT);  
    pinMode(rightMotorPin2,OUTPUT);  
  
    pinMode(enableLeftMotor,OUTPUT);  
    pinMode(leftMotorPin1,OUTPUT);  
    pinMode(leftMotorPin2,OUTPUT);  
  
    rotateMotor(0,0);  
  
    radio.begin();  
    radio.setDataRate(RF24_250KBPS);  
    radio.openReadingPipe(1,pipeIn);  
    radio.startListening(); //start the radio receiver  
  
    #ifdef PRINT_DEBUG  
    Serial.begin(115200);  
    #endif  
}
```

```
void loop()  
{  
    int rightMotorSpeed=0;  
    int leftMotorSpeed=0;  
    // Check if RF is connected and packet is available  
    if(radio.isChipConnected() && radio.available())  
    {  
        radio.read(&receiverData, sizeof(PacketData));  
        int mappedYValue = map(receiverData.yAxisValue, 0, 254, -255, 255);
```



```

int mappedXValue = map(receiverData.xAxisValue, 0, 254, -255, 255);
int motorDirection = 1;
if (mappedYValue < 0)
{
    motorDirection = -1;
}

rightMotorSpeed = abs(mappedYValue) - mappedXValue;
leftMotorSpeed = abs(mappedYValue) + mappedXValue;

rightMotorSpeed = constrain(rightMotorSpeed, 0, 255);
leftMotorSpeed = constrain(leftMotorSpeed, 0, 255);

rotateMotor(rightMotorSpeed * motorDirection, leftMotorSpeed * motorDirection);

lastRecvTime = millis();

#ifdef PRINT_DEBUG
Serial.println(receiverData.xAxisValue);
Serial.println(receiverData.yAxisValue);
#endif
}
else
{
    //Signal lost. Reset the motor to stop
    unsigned long now = millis();
    if ( now - lastRecvTime > SIGNAL_TIMEOUT )
    {
        rotateMotor(0, 0);
    }
}
}

void rotateMotor(int rightMotorSpeed, int leftMotorSpeed)
{
    if (rightMotorSpeed < 0)
    {
        digitalWrite(rightMotorPin1, LOW);
        digitalWrite(rightMotorPin2, HIGH);
    }
    else if (rightMotorSpeed > 0)
    {
        digitalWrite(rightMotorPin1, HIGH);
        digitalWrite(rightMotorPin2, LOW);
    }
    else
    {

```

```
digitalWrite(rightMotorPin1,LOW);
digitalWrite(rightMotorPin2,LOW);
}

if (leftMotorSpeed < 0)
{
digitalWrite(leftMotorPin1,LOW);
digitalWrite(leftMotorPin2,HIGH);
}
else if (leftMotorSpeed > 0)
{
digitalWrite(leftMotorPin1,HIGH);
digitalWrite(leftMotorPin2,LOW);
}
else
{
digitalWrite(leftMotorPin1,LOW);
digitalWrite(leftMotorPin2,LOW);
}
analogWrite(enableRightMotor, abs(rightMotorSpeed));
analogWrite(enableLeftMotor, abs(leftMotorSpeed));
}
```

PROBLEMS FACED

1. Calibration of the MPU6050 to accurately interpret hand gestures.
2. Ensuring a robust wireless connection between the hand controller and the car using the NRF modules.
3. Precision in recognizing gestures to avoid unintended car movements.
4. Optimization of code for efficient processing and responsiveness.
5. Overheating and melting of li-ion batteries.

CONCLUSION

In conclusion, the hand gesture-controlled car project utilizing the MPU6050, Arduino Nano, and NRF transceiver marks a significant stride at the intersection of sensor technology, wireless communication, and robotics.

Throughout this project, we've successfully harnessed the capabilities of the MPU6050 sensor to detect hand gestures by capturing variations in pitch, roll, and yaw, thereby enabling intuitive control mechanisms. Leveraging the Arduino Nano's processing prowess, we've translated these gestures into actionable commands for the car's movement, exhibiting a seamless interaction between human gestures and machine response.

The integration of the NRF transceiver facilitated wireless communication, enabling real-time transmission of gesture commands to the car, paving the way for untethered control and enhancing the project's usability and mobility.

Challenges encountered during the project, such as sensor calibration, gesture recognition accuracy, and maintaining a reliable wireless connection, were systematically addressed and currently being looked into to get resolved. Optimizations were implemented to ensure efficient data processing, improving the system's responsiveness and reliability.

This project not only highlights technical proficiency but also underscores the practical application of sensor fusion, wireless communication, and algorithmic interpretation in the realm of gesture-controlled robotics. Future iterations may focus on refining gesture recognition algorithms, enhancing the wireless communication protocol for greater reliability, and expanding the car's functionalities based on the detected gestures.

Ultimately, the hand gesture-controlled car project serves as a testament to the fusion of hardware, software, and innovation, showcasing the potential of gesture-based control systems in advancing human-machine interaction in robotics and automation.

BIBLIOGRAPHY

- [1] www.igniteinnovateideas.wordpress.co.in
- [2] www.researchgate.net
- [3] www.elprocus.com/accelerometer-based-gesture-control-robot/
- [4] www.create.arduino.cc/projecthub
- [5] J.Pradeep , Dr.P.Victor Paul” Design and Implementation of Gesture Control Robotic Arm for Industrial Applications”, International Journal of Advanced Scientific Research & Development ,vol.03,pp.202-209,Oct-Dec’2016
- [6] Image 1: Hand Gestured Controlled Car circuit layout Ref:
<https://create.arduino.cc/projecthub/mayooghgirish/handgesture-controlled-robot-4d7587>
- [7] Image 2: Circuit Diagram of Transmitter Ref:
<https://create.arduino.cc/projecthub/mayooghgirish/handgesture-controlled-robot-4d7587>
- [8] Image 3: Circuit Diagram of Receiver Ref:
<https://create.arduino.cc/projecthub/mayooghgirish/handgesture-controlled-robot-4d7587> Receiver Circuit
- [9] Image 4(a)Gesture to move left , 4(b) Gesture to move right ,4(c) Gesture to stop,4(d) Gesture to move forward,4(e) Gesture to move backward Ref:
<https://create.arduino.cc/projecthub/mayooghgirish/handgesture-controlled-robot-4d7587> left_flat , right_flat ,Stop_flat, ,forward_flat,backward_flat